

Data-Driven Optimisation for Machine Learning applications - Project report: DQL for QSERVO3

Noémie THOMASSON

TU Ilmenau – Juin 2026

Contents

1	Introduction	3
2	General fonctionning & Mechanical background	3
2.1	Physical Specification	3
2.2	General Idea	3
3	Swing up	3
3.1	Theory	3
4	Stabilization	4
4.1	Theory	4
4.1.1	Reinforcement Learning	4
4.1.2	Deep Q-Learning	4
4.1.3	Double Deep Q-Learning	5
4.2	Model Strategy	5
4.3	Agent	7
4.4	Parameters	7
5	Main Loop	7
5.1	Training	8
6	Results	8
6.1	Discussion	9

1 Introduction

This project aims to apply Deep-Q-Learning to a Quanser Qube Servo-3 in order to perform a **swing up** and **stabilization** of his pendulum. The code source of this project is available on Moodle as a zip file. This report tends to present the theory behind the project and how we concretized it.

2 General functioning & Mechanical background

2.1 Physical Specification

We work on the Qube-Servo-3 equipped with a pendulum composed of one arm of length l , with a rotation angle of θ , the pendulum of length h and rotation angle of α . We assume that when the pendulum is to the ground $\alpha = 0$ and when it is up, $\alpha = \pi$. On this device, the θ rotation is between $[-90^\circ, 90^\circ]$.

2.2 General Idea

The goal functionality is to perform a swing up of the pendulum followed by his stabilization. Therefore the project will be divided in two part: one concentrated on the **swing up** that only follows mechanical rules and another part on the **stabilization** performs by Deep-Q-Learning. The goal is to start the process by a swing-up, when α gets to a certain value, it switches to the stabilization process. If this process fails and α returns to a low value, the process switches back to a swing-up process.

3 Swing up

3.1 Theory

The swing-up process relies on mechanical formulas depending on the two angles θ and α and their angle velocity $\dot{\theta}$ and $\dot{\alpha}$.

For the swing-up process, we will follow the energy stabilization strategy. The goal is not to control the angle, or velocity but the energy. It is ruled by the following dynamic equations which are presented in the paper [2]:

$$E = \frac{1}{2}J\dot{\theta}^2 + mgl(\cos \theta - 1)$$

where:

m : pendulum mass

l : distance between pivot and mass center

J : inertia momentum

θ : pendulum angle

u : horizontal acceleration of the pivot

This global energy comes from the addition of potential energy, E_p and the kinetic energy, E_c . We then compare two energies: E , the actual energy, and E_d the desire energy:

$$e_d = E - E_d$$

When $e_d < 0$, it lacks energy, when $e_d > 0$, there is too much energy. Therefore, the objective is $e_d = 0$. According to the paper, the energy is deeply influenced by $\dot{\theta}\cos(\theta)$ due to the obtained time derivative.

We know that the direction of theta and how fast it moves has a big influence in our problem. The formula stated in the paper uses only the sign of the energy value, but we can use a linear augmentation to smooth the movement. We can then compute a "control law" based on these parameter and the energies differences as followed:

$$u = k(E - E_d)\dot{\theta}\cos(\theta)$$

where k is an amplification factor to be set.

4 Stabilization

4.1 Theory

4.1.1 Reinforcement Learning

Reinforcement Learning is a method of Deep Learning that, unlike Supervised Learning, doesn't need any previous data processing/labeling. The loss calculation is calculated depending on a **reward** metric. A reinforcement learning model collect data and learns continuously while interacting with the environment. The steps are as followed:

1. We do an action
2. We get consequences of our action
3. We calculate a reward metric depending on how good this action was for our goal.
4. We update the strategy for further actions

The reward metric is calculated as followed:

$$R_{t+1} = R(S_t, A_t, S_{t+1})$$

We can then compute a Q score value for the state and action:

$$Q^*(s, a) = R(s, a) + \gamma \max_{a'} Q^*(s', a')$$

This score is updated each time the agent takes a step. During execution, when in state **s**, we select the **a** value that maximizes the **Q** value.

A simple reinforcement learning with Q-Learning architecture represents limits as stated in this course: the Q table only takes discrete values of action, by discretization, the dimension of our table will increase significantly. The provided solution is to go from a Q-table to a weighted Neural Network.

4.1.2 Deep Q-Learning

Instead of looking up an exact row cell, we feed the raw continuous state vector **s** as an input into a neural network, and the network outputs (predicts) the Q-values for all available actions. We use the Mean Squared Bellman Error (MSBE) in which we find the terms of Q-Learning.

While learning, we compute a target value which associates with the score used in Q-Learning. This value

is what we would like the action to achieve. The target value is calculated as follow:

$$Target = R_{t+1} + \gamma max_{a'} Q(s_{t+1}, a_{t+1})$$

As we see here, the target is computed using the current weights that are being learned. However, since the network still evolves, this creates a paradox, when the target evolves while the network is trying to match it.

4.1.3 Double Deep Q-Learning

The architecture chosen as a solution to this problem is a *Double-DQL*. The idea of *Double Q-Learning* is to set a second network: the **target network** which is a copy of the initial network called **online network**, at some points of the training. The target network is only used during the **online network** optimization, when **y** is calculated. Instead of using **online network's Q** value, we use the target values of the **target network**.

4.2 Model Strategy

The model is a two fully connected linear layers of dimension **128** separated by ReLU activation function..

The model was train directly on hardware, using the swing up and to get to the top, where it handed the controle to DQN to learn. The DQN is activated only if the pendulum gets to a certain angle: $\pi \pm 15^\circ$ with a certain speed (7 rad/s).

At the end, this strategy is not the most effective and takes a long time since, for one episode, we need to wait that the pendulum get to the good state. On the other hand, the avantage is that the model is directly adapted to the hardware and need no fine-tuning. The other possibility would have been to use a simulation to train the stabilization starting the pendulum directly from the good position and speed. The issue is that the simulation has to match almost perfectly the real physics conditions.

The model pipeline is illustrated in this diagram:

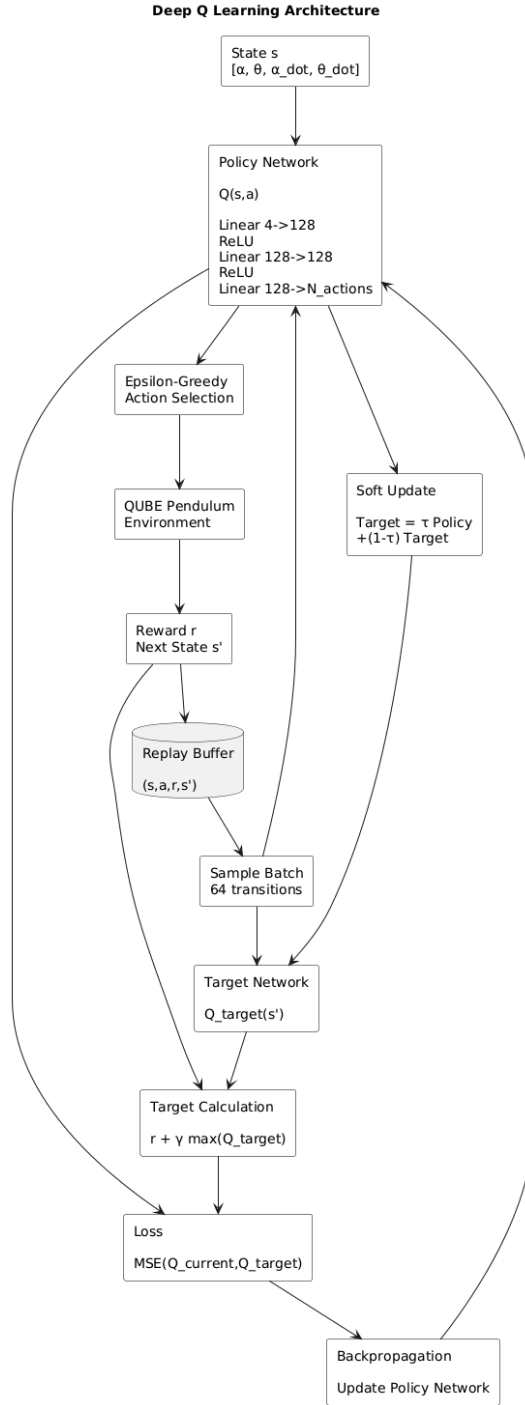


Figure 1: Model pipeline.

4.3 Agent

The agent chooses between a defined set of action and supervises at multiple times in the process the state of the servo. The states take into account θ , $\dot{\theta}$, α , $\dot{\alpha}$. The actions defined in this project are 21 possibles tension in the interval: $[-2, 2]$:

```
ACTIONS = np.linspace(-2, 2, 11)
```

Another key aspect of the training process is defining the right **reward computation**. The original reward takes into account the angles and their velocity. However, we are constrained by the $[-90^\circ, 90^\circ]$ angle interval. Therefore, in addition to other security, we penalize the reward if the pendulum goes to close 90° or -90° . Another bonus is added when the pendulum is stable upright and is alpha error is really small. The final reward is as followed:

```
reward = (
    10*np.cos(alpha_mod)
    -0.02*theta**2
    +stability_bonus
    -0.05*alpha_dot**2
    -0.001*theta_dot**2
    -0.0005*action_voltage**2
)
if abs(abs(alpha_mod)-np.pi) < np.deg2rad(15):
    reward += 80
if abs(theta)>math.radians(90):
    reward-=220
if abs(alpha_dot) > 5:
    reward -= 100
```

4.4 Parameters

Hyperparameters							
Parameter	Batch size	Actions	Nb of episodes (training)	Model dimension (hidden layer)	learning rate	γ	ϵ
Value	64	linspace(-5.8, 5.8, 21)	1000-2000	128	e-5	0.99	EPS_DECAY = 2500, EPS_END = 0.01, EPS_START = 0.9

5 Main Loop

The problem is divided in two phases: the swing up and the stabilization. They are performed by two different agents. The *SwingUp* perform a swing up considering the dynamic equations stated before. The *Stabilizer* try to stabilize the pendulum once it reaches the upright position, in a angle range around the top, using Deep Q-Learning. Therefore, the global process alternates between the SwingUp agent and the Stabilizer agent depending on the α angle and its velocity $\dot{\alpha}$. This processus of switching between agents

is used during the training and during the test.

5.1 Training

The order of actions is the following:

Algorithme 1 : Main loop

Connect to card; Initialize instances and variables;

for $N_EPISODE$ **do**

 get α , θ , $\dot{\alpha}$ $\dot{\theta}$;

 balance pendulum until $\alpha < \alpha_{threshold}$;

while *pendulum not upright stabilize* **do**

 select_action();

 apply_action();

 get_next_state();

 get_reward();

 is_done();

 update_memory();

 optimize_network();

 save_network_weights();

6 Results

The training products stabilization results but seem not to take into account enough the good scores. Therefore the reward doesn't converge enough to tend to a good score. Here is the best training's reward function per episode for 300 episodes:

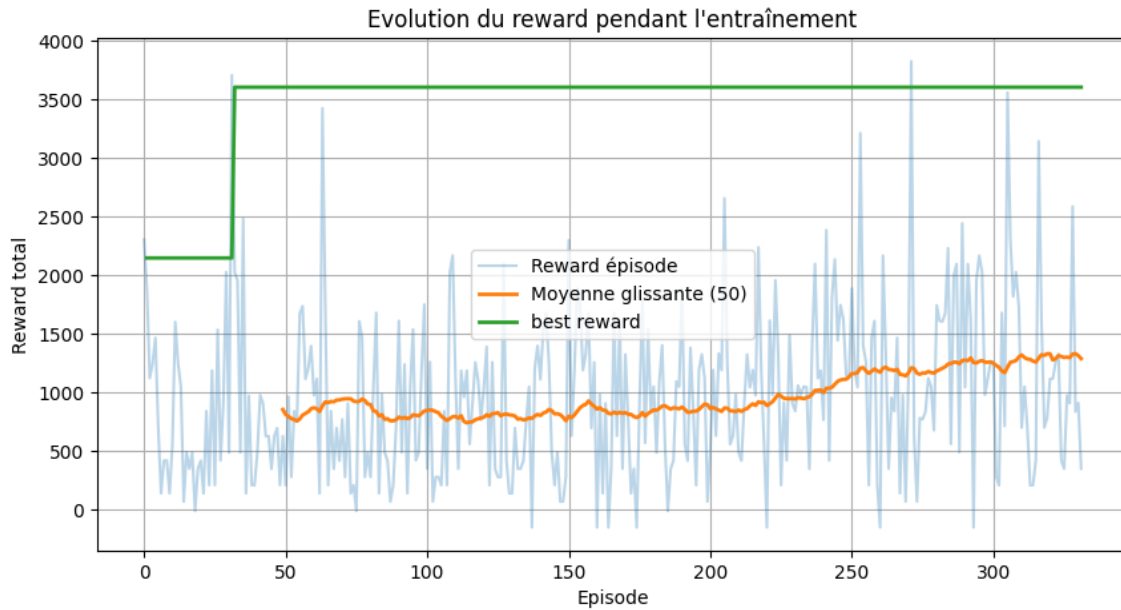


Figure 2: Model pipeline.

We observe that the model learns quickly in the start of the training and updates his best reward often; however the learning speed decreases with more episode and finally it stops learning. We can understand that the model is limited to a maximum learning state which shows the main problem of this project is architectural and not the number of training episode. This stagnation can also be caused by the motor of the servo heating up and therefore changing its constant.

6.1 Discussion

The training strategy used, with using the swing-up in the training, can also bring a lot of uncertainty. The swing-up has to be optimal, with minimal variation and speed when balancing the pendulum to the top. This could be avoided by using a simulation, that could cover more "up" situations (angle and speed). Moreover, this adds stochastic parameters on which the training depends on. Instead of relying only on the DQN parameter (actions, dimensions, etc), it has to adapt to the upcoming speed.

References

- [1] M. Levis, *Using the Reinforcement Learning Toolbox™ to Balance the Qube-Servo 3 Inverted Pendulum*, Quanser Blog, March 11, 2026.
Available online: <https://www.quanser.com/blog/artificial-intelligence/using-the-reinforcement-learning-toolbox-to-balance-the-qube-servo-3-inverted-pendulum/>
- [2] K. J. Åström and K. Furuta, *Swinging Up a Pendulum by Energy Control*, Automatica, vol. 36, no. 2, pp. 287–295, 2000.
DOI: 10.1016/S0005-1098(99)00140-5
- [3] S. Fayazi et al., *Reinforcement Learning Approach to Control an Inverted Pendulum: A General Framework for Educational Purposes*, PLOS ONE, vol. 18, no. 1, e0280071, 2023.
PDF available on HAL: <https://hal.science/hal-03999495/file/journal.pone.0280071.pdf>